

SMC Reset Unit — Low-Level Design

SMC Reset Unit — Low-Level Design (Internal)

This document describes the **internal SystemC implementation** of the SMC Reset Unit LT model: process structure, single-driver discipline, reset-domain handling, FLR sequencing, and the CCI parameter catalogue. The external contract is in `01_RESET_UNIT_Specification.md`.

1. Source structure

File	Contents
<code>include/reset_unit.h</code>	<code>reset_ctrl_t</code> , <code>jtag_reset_ctrl</code> , <code>reset_unit_cfg</code> , <code>SC_MODULE(reset_unit)</code> + <code>cci_param</code> declarations.
<code>src/reset_unit.cpp</code>	Constructor, processes, register decode, reset derivation, FLR sequencing, debug API.
<code>include/smc_tlm_extensions.h</code>	Shared <code>smc_axi_extension</code> GP extension (standalone build).

The RTL’s structural split (`smc_reset_ctrl` / `smc_subsystem_resets` / `smc_cool_reset_wrap` / `smc_reset_sync`) is collapsed into a single `SC_MODULE`, since an LT model has no need for the four separate clock domains.

2. Class structure & public API

`reset_unit` is a single `sc_core::sc_module`. Its CCI parameters are declared **before** the public ports (so they are constructed first and `ss_reset_ctrl_o` can be sized from `num_subsystems`). The tables below catalogue the complete interface; per-signal descriptions live in `include/reset_unit.h`.

2.1 Ports

Port	Type	Dir	Purpose
reg_socket	tlm_utils::simple_target_socket<reset_unit>	target	AXI-Lite-style 32-bit register access.
powergood_i	sc_core::sc_in<bool>	in	Power-good from pad (active-high).
rst_cold_ni	sc_core::sc_in<bool>	in	Cold reset from pad (active-low).
fuse_reset_ni	sc_core::sc_in<bool>	in	Fuse-sensing-done reset (active-low).
rst_ext_wdt_ni	sc_core::sc_in<bool>	in	External watchdog reset (active-low).
smc_wdt_first_timeout_i	sc_core::sc_in<bool>	in	WDT first timeout (informational).
smc_wdt_second_timeout_i	sc_core::sc_in<bool>	in	WDT second timeout (forces core/WDT reset).
rst_cool_ni	sc_core::sc_in<bool>	in	Incoming cool reset from primary chiplet (active-low).
isolate_req_pin_i	sc_core::sc_in<bool>	in	External isolate-request pin.
cfg_flr_pf_active_i	sc_core::sc_in<bool>	in	PCIe FLR active; rising edge starts cool-reset flow.
ss_reset_complete_i	sc_core::sc_in<uint32_t>	in	Per-subsystem reset-complete status.
captured_straps_i	sc_core::sc_in<uint64_t>	in	Captured GPIO straps (STRAPS_LO/HI).
powergood_stable_o	sc_core::sc_out<bool>	out	Stable (stretched) power-good.
rst_cold_stable_ref_clk_no	sc_core::sc_out<bool>	out	Stable cold reset, ref-clk domain.
rst_cold_stable_smc_clk_no	sc_core::sc_out<bool>	out	Stable cold reset, SMC-clk domain.
rst_primary_ref_clk_no	sc_core::sc_out<bool>	out	Primary reset, ref-clk domain.
rst_primary_smc_clk_no	sc_core::sc_out<bool>	out	Primary reset, SMC-clk domain (register-block reset).
rst_primary_periph_clk_no	sc_core::sc_out<bool>	out	Primary reset, peripheral-clk domain.
rst_core_smc_clk_no	sc_core::sc_out<bool>	out	Core reset, SMC-clk domain.
rst_wdt_smc_clk_no	sc_core::sc_out<bool>	out	WDT reset, SMC-clk domain.
rst_cool_no	sc_core::sc_out<bool>	out	Outgoing cool reset to secondary chiplets (active-low).
skip_mem_repair_o	sc_core::sc_out<bool>	out	Skip memory repair / MBIST during FLR.

Port	Type	Dir	Purpose
sync_irq_o	sc_core::sc_out<bool>	out	Global sync IRQ (SYNC_REG.bit0).
isolate_req_o	sc_core::sc_out<uint32_t>	out	Per-subsystem isolate request.
ss_config_o	sc_core::sc_out<uint32_t>	out	Subsystem configuration register value.
ss_reset_ctrl_o[i]	sc_core::sc_vector<sc_core::sc_out<reset_ctrl_t>>	out	Per-subsystem reset-control bundle; sized from num_subsystems .

2.2 Public methods (API)

Method	Signature	Purpose
constructor	reset_unit(sc_module_name name, reset_unit_cfg cfg = {})	Build, resolve CCI, size ss_reset_ctrl_o , register callbacks/processes (see §3).
set_jtag_ctrl	void set_jtag_ctrl(const jtag_reset_ctrl& j)	Apply a JTAG reset-override bundle and recompute outputs.
jtag_ctrl	const jtag_reset_ctrl& jtag_ctrl() const	Back-door view of the current JTAG override bundle.
dbg_read	uint32_t dbg_read(uint64_t off) const	Back-door register read by byte offset; no side effects, no delay.
dbg_ss_reset_ctrl	reset_ctrl_t dbg_ss_reset_ctrl(unsigned i) const	Back-door view of subsystem i 's reset-control bundle.
num_subsystems	unsigned num_subsystems() const	CCI-resolved subsystem count.
dump_state	void dump_state(std::ostream& = std::cout) const	Human-readable register file + derived-reset snapshot.

2.3 Internal methods and SC processes

Member	Signature	Kind	Purpose
<code>b_transport</code>	<code>void b_transport(tlm::tlm_generic_payload&, sc_core::sc_time&)</code>	TLM <code>b_transport</code> callback	Blocking 32-bit register data path (§9).
<code>transport_dbg</code>	<code>unsigned int transport_dbg(tlm::tlm_generic_payload&)</code>	TLM <code>transport_dbg</code> callback	Side-effect-free back-door register access (§9).
<code>input_method</code>	<code>void input_method()</code>	SC_METHOD (all input ports)	Track inputs, detect FLR edge, call <code>process_state_change()</code> (§4).
<code>flr_assert_method</code>	<code>void flr_assert_method()</code>	SC_METHOD (<code>flr_assert_event_</code>)	Drive <code>flr_cool_n_low</code> (§7).
<code>flr_deassert_method</code>	<code>void flr_deassert_method()</code>	SC_METHOD (<code>flr_deassert_event_</code>)	Drive <code>flr_cool_n_high</code> (§7).
<code>output_method</code>	<code>void output_method()</code>	SC_METHOD (<code>recompute_event_</code>)	Sole driver of every output port (§4).
<code>start_of_simulation</code>	<code>void start_of_simulation() override</code>	SystemC lifecycle hook	Seed reset-edge trackers; post the first recompute (§4).
<code>schedule_recompute</code>	<code>void schedule_recompute()</code>	private helper	Post <code>recompute_event_</code> at <code>SC_ZERO_TIME</code> (§4).
<code>reg_read</code>	<code>bool reg_read(uint64_t off, uint32_t& data) const</code>	private helper	Decode a register read; false if out-of-window (§9).
<code>reg_write</code>	<code>bool reg_write(uint64_t off, uint32_t data)</code>	private helper	Decode a register write; false if out-of-window (§9).
<code>derive</code>	<code>derived_t derive() const</code>	private helper	Combinationally derive all reset levels (§5).
<code>process_state_change</code>	<code>void process_state_change()</code>	private helper	Re-derive, clear reset-domain register groups on assert edges, recompute (§6).
<code>clear_primary_regs</code>	<code>void clear_primary_regs()</code>	private helper	Clear the primary-domain register group (§6).
<code>clear_cold_regs</code>	<code>void clear_cold_regs(bool isolate_pin)</code>	private helper	Clear the cold-domain (FLR/isolate) register group (§6).
<code>flr_kick</code>	<code>void flr_kick()</code>	private helper	Schedule the FLR cool-reset pulse events (§7).

Processes/threads: the model registers exactly **four** SC processes — `input_method`, `flr_assert_method`, `flr_deassert_method`, and `output_method`, all **SC_METHOD** and all `dont_initialize()` d. There are **no** **SC_THREAD** / **SC_CTHREAD** processes; `output_method` is the only writer of any `sc_out` (single-driver discipline, §4).

2.4 File-local helpers (`reset_unit.cpp` anonymous namespace)

Function	Signature	Purpose
<code>bit</code>	<code>constexpr bool bit(uint32_t v, unsigned i)</code>	Extract bit <code>i</code> from a 32-bit register/mask value.

3. Constructor

```

reset_unit::reset_unit(sc_core::sc_module_name name, reset_unit_cfg cfg)
: sc_core::sc_module(name)
// (1) CCI params constructed first (declared before the public ports).
//    cfg.* values are the DEFAULTS; broker presets override them.
, num_subsystems_p_ ("num_subsystems",    cfg.num_subsystems,    "...")
, ref_clk_period_ns_p_ ("ref_clk_period_ns", cfg.ref_clk_period_ns, "...")
, access_delay_ns_p_ ("access_delay_ns",    cfg.access_delay_ns,    "...")
// (2) Ports. ss_reset_ctrl_o is sized from the (possibly preset)
//    num_subsystems value resolved above.
, reg_socket("reg_socket")
, powergood_i("powergood_i") /* ... all scalar in/out ports named ... */
, ss_reset_ctrl_o("ss_reset_ctrl_o", num_subsystems_p_.get_value())
, cfg_(cfg)
{
    // (3) Sync cfg_ with the (possibly preset-overridden) CCI values.
    cfg_.num_subsystems = num_subsystems_p_.get_value();
    cfg_.ref_clk_period_ns = ref_clk_period_ns_p_.get_value();
    cfg_.access_delay_ns = access_delay_ns_p_.get_value();

    // (4) Provenance metadata for introspection tools.
    num_subsystems_p_.add_metadata("rdl_dimension",
        cci::cci_value(std::string("ss_reset_ctrl_o[31:0]")));
    num_subsystems_p_.add_metadata("valid_range", cci::cci_value(std::string("1..32")));
    ref_clk_period_ns_p_.add_metadata("unit", cci::cci_value(std::string("nanoseconds")));
    ref_clk_period_ns_p_.add_metadata("rtl_signal", cci::cci_value(std::string("clk_ref_i")));
    access_delay_ns_p_.add_metadata("unit", cci::cci_value(std::string("nanoseconds")));
    access_delay_ns_p_.add_metadata("tlm_phase", cci::cci_value(std::string("annotated_delay")));

    // (5) Validate – fail loud at elaboration, not at first transaction.
    if (cfg_.num_subsystems == 0 ||
        cfg_.num_subsystems > reset_unit_cfg::MAX_SUBSYSTEMS)
        SC_REPORT_FATAL(name, "reset_unit num_subsystems must be in 1..32");
    if (cfg_.ref_clk_period_ns < 0.0)
        SC_REPORT_FATAL(name, "reset_unit ref_clk_period_ns must be >= 0");
    if (cfg_.access_delay_ns < 0.0)
        SC_REPORT_FATAL(name, "reset_unit access_delay_ns must be >= 0");

    // (6) Size the per-subsystem idempotence cache; cache the time-typed delays
    //    (access_delay_ is re-cached on every b_transport so a CCI mutation
    //    of access_delay_ns takes effect on the next access).
    ss_ctrl_cache_.assign(cfg_.num_subsystems, reset_ctrl_t{});
    access_delay_ = sc_core::sc_time(cfg_.access_delay_ns, sc_core::SC_NS);
    ref_clk_period_ = sc_core::sc_time(cfg_.ref_clk_period_ns, sc_core::SC_NS);

    // (7) Register the TLM target-socket callbacks.
    reg_socket.register_b_transport (this, &reset_unit::b_transport);
    reg_socket.register_transport_dbg(this, &reset_unit::transport_dbg);

    // (8) SC_METHODs (all dont_initialize()'d; start_of_simulation seeds them).
    SC_METHOD(input_method); // sensitive to every input port
    sensitive << powergood_i << rst_cold_ni << /* ... */ << captured_straps_i;
    dont_initialize();

    SC_METHOD(flr_assert_method); sensitive << flr_assert_event_; dont_initialize();
    SC_METHOD(flr_deassert_method); sensitive << flr_deassert_event_; dont_initialize();

    SC_METHOD(output_method); // the only writer of any sc_out
    sensitive << recompute_event_;
    dont_initialize();

    // (9) Human-readable "instantiated with ..." banner.
    SC_REPORT_INFO(name, /* num_subsystems / ref_clk_period_ns / access_delay_ns */ ...);
}

```

Arguments

Parameter	Type	Default	Role
name	sc_core::sc_module_name	—	SystemC instance name; also the report context for SC_REPORT_*.
cfg	reset_unit_cfg	reset_unit_cfg{}	Sizing / timing defaults (num_subsystems, ref_clk_period_ns, access_delay_ns, and the fixed register offsets). Any matching CCI broker preset overrides the corresponding cfg field.

The constructor never reaches the first transaction with an invalid configuration: all validation runs at elaboration via SC_REPORT_FATAL. The four SC_METHODs are registered but dont_initialize() d, so the model's outputs are first driven from start_of_simulation() (§4), which seeds the reset-edge trackers from the initial pin levels and posts recompute_event_.

4. Process structure

Process	Kind	Sensitivity	Role
input_method	SC_METHOD	all input ports	Track ss_reset_complete_i; detect FLR rising edge; call process_state_change().
flr_assert_method	SC_METHOD	flr_assert_event_	Drive flr_cool_n_low (cool reset asserted).
flr_deassert_method	SC_METHOD	flr_deassert_event_	Drive flr_cool_n_high (cool reset released).
output_method	SC_METHOD	recompute_event_	Sole driver of every output port.

All input/output processes are dont_initialize() d; start_of_simulation() seeds the reset-edge trackers from the initial pin levels and posts the first recompute_event_ so the outputs reach their reset-correct values at t=0.

Single-driver discipline

output_method is the only process that writes any sc_out. Every other code path (register write, FLR edge, JTAG override, input change) mutates internal state and calls schedule_recompute(), which posts recompute_event_ at SC_ZERO_TIME. This satisfies SystemC 3.0's strict single-driver rule and mirrors the PLIC / CLINT models. Output writes are **idempotent** for the per-subsystem bundles (cached in ss_ctrl_cache_) to avoid spurious value-changed events on the 32 struct signals.

5. Reset derivation (derive())

derive() is a pure function of the current input ports, the JTAG override bundle, and flr_cool_n_. It reproduces the smc_reset_ctrl.sv combinational equations exactly (see spec §5) but **abstracts the timing elements**:

RTL element	LT abstraction
32-cycle cold-reset de-glitch shift register	zero latency
255-cycle cold-reset extender counter	zero latency
32-cycle powergood stretch	<code>powergood_stable = powergood_i</code>
32-cycle cool-from-pin de-glitch	zero latency
<code>prim_sync_reset</code> (4-deep) synchronisers	zero-delay pass-through

These are CDC / glitch-filtering details that do not change the functional (firmware-observable) reset relationships.

The `derived_t` struct returned by `derive()` carries the four levels the rest of the model needs: `powergood_stable`, `stable_cold_rst_n` (= cold domain), `rst_primary_n` (= primary domain), `rst_core_int_n`, `rst_wdt_n`.

6. Register state & reset domains

Registers are plain `uint32_t` members grouped by reset domain (`clear_primary_regs()` / `clear_cold_regs()`). `process_state_change()`:

1. Calls `derive()`.
2. On a **falling edge** of `rst_primary_n` (tracked via `prev_primary_n_`), clears the primary-domain group (`SS_WARM_RESET_N` → `0xFFFFFFFF`, rest 0).
3. On a **falling edge** of `stable_cold_rst_n` (tracked via `prev_cold_n_`), clears the cold-domain group. `ISOLATE_REQ_PINEN_REG` is cleared only when `isolate_req_pin_i` is low (the RTL gates its reset with the pin).
4. Updates the trackers and calls `schedule_recompute()`.

Because `rst_primary_n` depends on `stable_cold_rst_n`, a cold reset asserts both domains; a cool/FLR reset (driven through `stable_cool_rst_n` / `flr_cool_n_`) asserts only the primary domain, so the FLR / isolate registers survive — exactly as the RTL retains the FLR counters across a cool reset.

`process_state_change()` is invoked from `input_method`, the two FLR methods, and `set_jtag_ctrl()`, so a reset assertion arising from any of those sources clears the right registers.

Known simplification. Register writes are not blocked while a reset is asserted (the RTL's register block ignores writes during `arst_n` low). Firmware does not write the block during reset, so the LT model permits it; documented here as a future-work item if strict write-gating is required.

7. FLR cool-reset sequencing

`input_method` detects the `cfg_flr_pf_active_i` rising edge (`prev_flr_active_`), sets `ISOLATE_REQ_SMC_REG.bit0`, and calls `flr_kick()`:

```
if (FLR_RESET_COUNTER_VALUE == 0) return;           // RTL: 0 ⇒ no flow
flr_assert_event_.notify(ref_clk_period × FLR_COUNTER_VALUE);
flr_deassert_event_.notify(ref_clk_period × (FLR_COUNTER_VALUE + FLR_RESET_COUNTER_VALUE));
```

`flr_assert_method` / `flr_deassert_method` then drive `flr_cool_n_` low/high and re-run `process_state_change()` (so the cool reset both pulses `rst_cool_no` and asserts the primary domain). This collapses the RTL's two `prim_updown_counter` + FSM pair into two timed events while preserving the observable pulse shape and timing (in `ref_clk_period_ns` units).

8. CCI parameter catalogue

Name	Type	Default	Mutability	Notes
<code>num_subsystems</code>	unsigned	32	<code>CCI_IMMUTABLE</code>	Sizes <code>ss_reset_ctrl_o[]</code> ; validated 1..32.
<code>ref_clk_period_ns</code>	double	10.0	<code>CCI_IMMUTABLE</code>	FLR pulse time base; validated ≥ 0 .
<code>access_delay_ns</code>	double	2.0	mutable	Re-read every <code>b_transport</code> ; validated ≥ 0 .

CCI params are declared **before** the public ports in the header so they are constructed first and `ss_reset_ctrl_o` can be sized from the (possibly preset-overridden) `num_subsystems`. Provenance metadata (`rdl_dimension`, `valid_range`, `unit`, `rtl_signal`, `tlm_phase`) is attached for inspector tooling. Misconfiguration (`num_subsystems` $\notin$ 1..32, negative periods) raises `SC_REPORT_FATAL` at construction.

9. TLM-2.0 callbacks

`b_transport` validates width (4 B), alignment, streaming width, and byte-enable absence, then window membership, before dispatching to `reg_read` / `reg_write` and annotating the delay with `access_delay_ns`. `transport_dbg` shares the decode but skips the delay and applies the same register semantics without simulation side effects (it does not run the FLR edge detector). DMI is never granted.

10. Future work

- Strict register write-gating during reset assertion.
- Optional cycle-accurate de-glitch / extender / stretch counters behind a CCI flag, for users who need the exact reset-stretch timing.
- VCD tracing helper for the `reset_ctrl_t` bundle signals.